

DSA Day 23: BST - deletion (all cases)

Book: Data Structures Through C in Depth - Ch6 §6.12.5 · pp 208-214 | Code in Python

Overview

Delete a leaf directly; a one-child node by splicing; a two-child node by replacing with its inorder successor.

Key concepts to master

- Three cases
- Inorder successor/predecessor

Python implementation

```
def delete(root,v):
    if not root: return None
    if v<root.data: root.left=delete(root.left,v)
    elif v>root.data: root.right=delete(root.right,v)
    else:
        if not root.left: return root.right
        if not root.right: return root.left
        s=root.right
        while s.left: s=s.left
        root.data=s.data; root.right=delete(root.right,s.data)
    return root
```

Complexity

$O(h)$.

Common pitfalls

- Not reconnecting children
- Wrong successor choice

Practice

- Delete a leaf, a one-child node, the root
- Why the inorder successor?

